Aliaksei Ivanou

brightbox

SENIOR SOFTWARE ENGINEER

8+ years experience

Summary:

Specializing in real-time C++ for embedded Linux and aerospace systems. The current focus is a SIP video phone platform: on the device side, peripheral drivers on a Yocto image, V4L2 capture and GStreamer media services in C++17 and a Qt Widgets call UI; on the server side, a Windows configuration and provisioning application on .NET 8. Background in satellite imagery processing and 3D visualization with OpenGL and GLSL, covering large raster rendering, atmosphere and lighting models and orbit display integrated with STK, all delivered under ECSS practice and MISRA C++. Earlier UAV work included FreeRTOS firmware on STM32 for sensor synchronization, MAVLink, sensor fusion and LiDAR-based 3D reconstruction with Open3D and OpenSceneGraph. Known for taking ownership of hardware-software integration problems, holding standards in cross-team code review and mentoring engineers new to real-time work.

Domains:

- Telecommunications
- Aerospace Systems
- Unmanned Aerial Vehicles

Programming Languages:

C++, C, C#, GLSL, JavaScript, HTML, CSS.

Operating Systems:

Linux, Windows, FreeRTOS.

Tools:

Yocto Project, Docker, GitLab CI, systemd, CMake, qmake, gcc, GDB, Valgrind, Visual Studio.

Frameworks & Libraries:

.NET 8, .NET Framework, WinForms, Qt, Qt Widgets, QSS, GStreamer, V4L2, Node.js, GDAL, GTest, Qt Test, xUnit.

Graphics and 3D:

OpenGL 4.x, GLSL, compute shaders, OpenSceneGraph, Open3D, PCL, STK.

Parallel computing:

OpenMP, SIMD (SSE, AVX2).

Multimedia:

WebRTC, Linphone-SDK, liblinphone, libyuv, mediasoupclient, libwebrtc,.

Databases:

SQLite.

Network:

SIP, SDP, RTP, SNMP, DHCP, TFTP, MAVLink.

Security:
OpenSSL, TLS, AES.

Navigation and Sensor Fusion:
RTK GNSS, IMU, EKF, PPS time synchronization.

Standards:
MISRA C++:2008, ECSS-E-ST-40C, ECSS-Q-ST-80C.

Embedded:
STM32, BitBake, NXP i.MX8M Mini.

Development Processes:
Agile, TDD, UML, Jira, Confluence.

Source control systems:
Git, GitLab, Bitbucket, SVN.

Employment History

SEPTEMBER 2023 – Till now, Software Engineer | SIP-BASED PHONE SOFTWARE

Description:
A SIP desk phone platform for audio and video communication, supporting both camera-enabled and audio-only device models. The project covers the embedded Linux image and phone application on the device, customized telephony and media components, a configuration application for administrators, web management and automated provisioning. It supports everyday calling workflows such as SIP registration, video calls, hold, transfer, conferencing, phonebook access and remote configuration, giving enterprise customers a centrally managed device fleet.

- Worked across the device stack, from Yocto/Linux and hardware integration through customized Linphone middleware to the Qt call UI;
- Refactored SIP call handling in the Qt Widgets phone application toward a cleaner MVC/MVVM-style structure, separating UI rendering from call state, user actions and SDK/media state;
- Documented the refactored call-handling architecture with UML class and sequence diagrams, covering the interaction between UI, call state and SDK/media layers;
- Integrated video calling and WebRTC conferencing on the ARM64 device, working with Linphone media callbacks, camera selection, libyuv frame conversion and mediasoupclient/libwebrtc rendering paths;
- Optimized scheduling and buffer management in the media path to reduce latency spikes and prevent audio/video degradation under conferencing load;
- Wrote and maintained Linux drivers for device peripherals such as the handset hook switch, keypad backlight and camera privacy shutter;
- Extended Linphone-SDK (liblinphone) backend logic to implement SIP failover and backup-server connectivity for improved device reliability;

- Developed WebRTC video-conferencing flows: conference creation and join, incoming invite screens, participant roster, video tile layout and audio/video device controls;
- Built the C# configuration application that administrators run on the server PC to prepare device profiles, roll out firmware and settings to phone fleets over SNMP, DHCP and TFTP, and keep device inventory in SQLite;
- Migrated the WinForms provisioning application from .NET Framework 4.8 to .NET 8, replacing legacy BinaryFormatter project persistence with explicit JSON state models while preserving firmware-facing provisioning output and MSTest coverage;
- Unified configuration file formats across device models, so every phone model read the same profile structure;
- Set up Docker-based CI in GitLab with a reproducible Yocto/BitBake build container, running Qt Test suites on every merge request;
- Applied TDD to call-state and provisioning logic, writing Qt Test and xUnit cases before implementation so regressions were caught on every merge request;
- Mentored two engineers joining the device team on real-time Linux and the media path, and led code reviews across the device and tooling teams;
- Worked in an Agile team, taking part in sprint planning, stand-ups and retrospectives, and maintained technical documentation for drivers, provisioning workflows and configuration profiles in Confluence.

Environment:
C++, C#, .NET 8, .NET Framework, WinForms, Qt, Qt Widgets, QSS, Linux, Windows, Yocto, BitBake, GStreamer, V4L2, Linphone-SDK, liblinphone, libyuv, mediasoupclient, libwebrtc, WebRTC, SIP, SDP, RTP, SNMP, DHCP, TFTP, SQLite, JavaScript, UML, HTML, CSS, TDD, Node.js, systemd, CMake, qmake, gcc, GDB, Valgrind, Qt Test, xUnit, Docker, GitLab CI, NXP i.MX8M Mini, Agile, Jira, Confluence, Git, GitLab.

MARCH 2020 – SEPTEMBER 2023, Software Engineer | SATELLITE IMAGERY PROCESSING AND 3D VISUALIZATION PLATFORM

Description:
A ground segment platform that turns raw Earth observation imagery into seamless, radiometrically consistent mosaics and presents them in an interactive 3D globe view alongside satellite orbits. Analysts and mission planners use it to inspect large survey areas, check imaging coverage and evaluate image quality before data is released to customers. The software was built to space-sector quality requirements, with verified accuracy of every processed product.

- Developed registration and mosaicking of overlapping satellite strips using tie-point matching, radiometric normalization and seam blending;
- Implemented spatial resolution assessment using edge targets, deriving MTF/RER metrics used by QA to verify image sharpness for each imaging campaign;
- Rendered multi-gigapixel rasters in OpenGL 4.x with a quadtree LOD scheme and asynchronous tile streaming, keeping the globe view interactive on analyst workstations;
- Wrote GLSL shaders for real-time visualization of cloud masks and raster overlays;
- Visualized satellite orbits and imaging swaths in 3D, pulling ephemeris and access data from STK through its Connect interface;
- Parallelized the processing chain with OpenMP and SIMD intrinsics (SSE, AVX2), bringing a full-scene mosaic run down from several hours to under 40 minutes;
- Secured image transfer with TLS/OpenSSL and protected stored imagery with AES encryption;
- Developed safety-critical C++ components under MISRA C++:2008 coding rules;
- Worked within the project-tailored ECSS-E-ST-40 / ECSS-Q-ST-80 software engineering and product-assurance processes;
- Produced software design documentation to ECSS requirements, including UML models of the processing chain and rendering components;
- Took part in peer code reviews to check compliance with MISRA C++:2008 rules and project coding standards;

- Validated geometric accuracy against reference ground control points, backing it with GTest regression suites that caught accuracy drift between releases.

Environment:
C++, OpenGL 4.x, GLSL, STK, OpenMP, SIMD (SSE, AVX2), GDAL, OpenSSL, TLS, AES, Linux, Windows, CMake, Visual Studio, GTest, MISRA C++:2008, ECSS-E-ST-40C, ECSS-Q-ST-80C, UML, Jira, Confluence, SVN.

SEPTEMBER 2018 – MARCH 2020, Software Engineer | UAV DATA PROCESSING AND 3D RECONSTRUCTION SOFTWARE

Description:
A software platform for UAV-based surveying of large objects such as buildings and industrial facilities. Onboard firmware synchronizes camera, IMU and LiDAR data, while the processing software stabilizes video, fuses sensor streams and turns captured scans into dense point clouds and 3D models. A Software-in-the-Loop environment reproduces flights on the bench, giving surveying and construction teams reliable spatial data without manual on-site measurement.

- Developed FreeRTOS firmware on STM32 for sensor synchronization, timestamping camera, IMU and LiDAR data on a hardware PPS signal from the GNSS receiver;
- Implemented video stabilization and debayering of raw camera frames, keeping the onboard preview in real time;
- Fused RTK GNSS and IMU data in an error-state Kalman filter to reconstruct the flight trajectory, deskewing each LiDAR scan line against it;
- Built the 3D reconstruction stage on Open3D and PCL, generating dense point clouds of buildings and industrial objects and rendering them for review;
- Replaced mutex-based hand-offs in the processing pipeline with lock-free ring buffers, removing frame drops at peak sensor rates;
- Built an OpenSceneGraph-based Software-in-the-Loop environment that replayed simulated flight paths and sensor scenarios, allowing payload software to be validated on the bench before flight tests;
- Emulated LiDAR sensors inside the SIL rig, generating synthetic returns with configurable noise and dropout models;
- Paired with the flight control and surveying teams to agree sensor mounting, capture patterns and acceptance criteria for each scan campaign;
- Developed processing modules test-first with GTest, validating sensor fusion and reconstruction stages against recorded and SIL-generated data within Agile iterations.

Environment:
C++, C, FreeRTOS, STM32, MAVLink, OpenSceneGraph, OpenGL, GLSL, compute shaders, Open3D, PCL, RTK GNSS, IMU, EKF, PPS time synchronization, Linux, CMake, gcc, GDB, GTest, Jira, Confluence, Git, Bitbucket, Agile.